

CAN VISION-LANGUAGE MODELS LEARN TO EXECUTE CLASSICAL SEARCH ALGORITHMS?

Anonymous authors

Paper under double-blind review

ABSTRACT

Prior work distinguishes operational reasoning from structural enumeration, which remains comparatively flat under scaling and prompting. We ask whether process supervision can teach a vision-language model to execute a declared search algorithm across full budgeted episodes. A Search Process Policy emits Typed Search Operations carrying every exploration-determining operand, while a Trusted Search Runtime validates and applies them without repair. Learned control is then measurable against an oracle-assisted, validity-constrained random-valid control that samples only runtime-valid operations. On a frozen 15-task text-state development panel for best-first width search, one fixed-seed process-supervised run achieved 0.933 success, versus 0.467 for random-valid, 0.0 for the base, and 1.0 for exact execution. Elsewhere, on saturated breadth-first and additive panels and a broader 24-task three-modality evaluation, process supervision showed no consistent advantage over random-valid across algorithms or modalities. All current results are single-seed and development-stage, with held-out primary evaluation, cross-backbone outcomes, generalization, and transfer unmeasured. Process supervision produced a learned-control advantage in one development setting, but broader planning efficacy remains open.

1 INTRODUCTION

Classical planning requires finding a sequence of actions that transforms an initial state into a goal state under specified constraints—a search problem whose formal structure allows instances to be generated systematically, solutions verified automatically, and difficulty controlled precisely, making it a natural probe of reasoning in foundation models (Valmeekam et al., 2023; Stein et al., 2025; Kokel et al., 2026). The formal model behind each instance fixes the task semantics, so errors reflect reasoning rather than ambiguity in the problem statement. The explicit state space also supports two levels of question: how actions apply locally, and how states connect globally. These properties let an evaluator vary difficulty along controlled dimensions and check each answer mechanically.

A recent study used this setting to measure latent planning competence in off-the-shelf language models (Huang et al., 2026). It separated two competencies that did not compensate for each other. Operational reasoning covers local applicability, progression, and validation: whether an action is legal in a state, what state it produces, and whether a goal condition holds. Structural enumeration covers goal reachability, landmarks, and other global state-space structure. Across model scales, and under chain-of-thought prompting, targeted prompting, and scratchpad or agent harnesses, operational reasoning improved, but these gains were not accompanied by comparable structural gains. The evidence was item-level and text-only, drawn from models used as released, without task-specific training.

Structural enumeration is related to global search-space reasoning, but it is not identical to full search execution. Identifying which states are reachable, or which serve as landmarks, does not require controlling a frontier across a budgeted episode. The prior study established this profile observationally. It did not test whether the structural competency can be acquired through training, or whether a model can sustain a declared procedure over a complete episode. Closing that gap calls for direct intervention: train the model to make the exploration decisions of a classical search algorithm, then measure whether structural/process competence changes.

Our intervention replaces direct plan generation with learned execution of a declared search algorithm. A Search Process Policy is trained to emit Typed Search Operations, discrete steps such as expanding a state, generating a successor, updating the frontier, or applying a goal test. A Trusted Search Runtime validates and applies each emission, checks Algorithm Invariants on every operation, and maintains bounded external Search Memory holding frontier, visited, novelty, and priority state. Ownership is replay-determinative. The emission must contain every operand that determines exploration, and the runtime may not select omitted operands, reorder candidates for the policy, or silently repair an invalid emission. Replaying the raw emissions against the same task and Search Memory must reproduce the explored trace. Operation validity, invariant compliance, and episode success are measured separately from learned control advantage, which is computed against an oracle-assisted, validity-constrained random-valid control that samples only runtime-valid operations.

The study asks whether a policy can learn to execute declared algorithms with valid, invariant-compliant traces, whether learned execution yields control advantage over the random-valid control, and whether the acquired capability transfers to planning benchmarks and broader search-intensive reasoning. The active algorithms are BFS, a complete unpruned width-based best-first search (BFWS), and two additive best-first variants, weighted and greedy. Process supervision on search-trace segments is the core training condition, and two-iteration DAgger, model-generated successors, and curriculum-by-modality have also been executed. Modality conditions present the same task content as text-state, visual-state, or multimodal-state observations under equal Search Memory capacity. References are the pretrained base, the random-valid control, and exact classical execution. Generalization and robustness checks, second-backbone evaluation, transfer, and the authorized final primary evaluation remain unmeasured.

The results form a headroom-dependent map. On a frozen 15-task text-state development panel under BFWS, one fixed-seed process-SFT run achieved 0.933 success, versus 0.467 for random-valid, 0.0 for the base, and 1.0 for exact execution. This is the only clear positive learned-control comparison, and it is development-stage. On BFS and the two additive best-first development panels, process SFT matched random-valid at 100%, so saturation left no room for measured advantage. An expanded baseline of 24 tasks across 12 domains and all three modalities reinforces the boundary: process SFT succeeded on 125 of 288 instances versus 240 for random-valid, 0 for the base, and 288 for exact execution, with every success in additive best-first cells, while BFS and width-based best-first failed on all 24 tasks in every modality. Two DAgger iterations added no advantage, and replacing trusted successors with model-generated ones dropped goal success from 45/45 to 1/45. All of these findings are single-seed and development-stage. The held-out primary evaluation, generalization, second-backbone outcomes, and transfer remain unmeasured.

This paper makes three contributions.

- **From diagnosis to intervention.** We execute the training-based follow-up to a diagnosed structural bottleneck: rather than prompt around flat structural enumeration, we train models to execute declared search algorithms and map where learned control emerges.
- **A replay-determinative policy/runtime interface.** Typed Search Operations carry every exploration-determining operand, while the no-repair Trusted Search Runtime validates and applies them. Operation validity, invariant compliance, episode success, and learned control advantage remain separate measurements.
- **A headroom-dependent empirical map.** Across development panels, learned control advantage appears only where saturation leaves headroom: a positive BFWS comparison, ceiling-matched BFS and additive best-first results, cross-algorithm and cross-modality failures in an expanded baseline, and negative DAgger and model-generated-successor boundaries. This is development-stage evidence, not final efficacy.

The paper therefore maps conditional, development-stage learnability of executable search rather than establishing general planning efficacy, with the final held-out evaluation, second-backbone outcomes, generalization, and transfer still unresolved.

REFERENCES

Sukai Huang, Chenyuan Zhang, Fucui Ke, Zhixi Cai, Naim Rastgoo, Gholamreza Haffari, and Hamid Reza Tofighi. What we talk about when we talk about LLM planning: Evidence for two

distinct planning abilities, July 2026.

Harsha Kokel, Michael Katz, Kavitha Srinivas, and Shirin Sohrabi. ACPBench Hard: Unrestrained reasoning about action, change, and planning. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, 2026.

Katharina Stein, Daniel Fišer, Jörg Hoffmann, and Alexander Koller. Automating the generation of prompts for LLM-based action choice in PDDL planning. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 35, pp. 250–259, 2025.

Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models—a critical investigation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, volume 36, pp. 75993–76005, 2023.

A APPENDIX

The Appendix will retain the complete evidence contract, including operation schemas, Algorithm Invariant definitions, runtime pseudocode, data and split procedures, receipt tables, statistics, adapters, and narrowly scoped feasibility details of retained infrastructure.